

JMAG Version 25.1

User's Manual

User-Defined Template API Functions and Samples

JSOL Corporation

■ COPYRIGHTS

Copyright © 2026 JSOL Corporation All Rights Reserved.

- Information in this document is subject to change without notice.
- No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or any means electronic or mechanical, including photocopying and recording for any purpose other than the purchaser's personal use without the written permission of JSOL Corporation.

JMAG Business Company
JSOL Corporation

Tokyo/ KUDAN-KAIKAN TERRACE 11th Floor, 1-6-5 Kudanminami, Chiyoda-ku, Tokyo 102-0074, Japan
Phone: +81(03)6261-7361 Phone (Support): +81(03)6261-7362 FAX: +81(03)5210-1142

Osaka/ Tosabori Daibiru Bldg. 2-2-4 Tosabori, Nishi-ku, Osaka 550-0001, Japan
Phone: +81(06)4803-5820 FAX: +81(06)6225-3517

Email: jmag-support@sci.jsol.co.jp URL: <https://www.jmag-international.com/>

■ Trademarks

- JMAG and JMAG-RT are registered trademarks of JSOL Corporation.
- Abaqus, CATIA, SIMULIA and SOLIDWORKS are trademarks or registered trademarks of Dassault Systemes or its subsidiaries in the United States and/or other countries.
- AMD is a trademark and AMD Opteron is a registered trademark of Advanced Micro Devices, Inc.
- Amesim, LMS Virtual.Lab, NX, Parasolid, Siemens, Simcenter, Simcenter 3D, Solid Edge, and STAR-CCM+ are trademarks or registered trademarks of Siemens Industry Software Inc., or its subsidiaries or affiliates, in the United States and in other countries.
- CAPRI CAE Gateway is a registered trademark of CADNexus Inc. in the United States and other countries.
- Creo and Pro/ENGINEER are registered trademarks of PTC, inc.
- DXF is a trademark and Inventor is a registered trademark of Autodesk, Inc. in the United States and/or other countries.
- Excel, Microsoft, Windows, Windows 10, and Windows 11 are registered trademarks of Microsoft Corporation in the United States and/or other countries. Microsoft Visual C++ is a registered trademark of Microsoft Corporation in the United States and/or other countries.
- Intel and Intel Xeon are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the United States and other countries.
- LabVIEW is a registered trademark of National Instruments Corporation.
- Linux is a registered trademark of Linus Torvalds in the United States and other countries.
- LSF is a registered trademark of IBM Corporation in the United States, other countries, or both.
- MATLAB and Simulink are registered trademarks of The MathWorks, Inc.
- MSC NASTRAN is a registered trademark of MSC Software Corporation, a subsidiary of Hexagon AB.
- NASTRAN is a registered trademark of the National Aeronautics and Space Administration.
- NX Nastran is an enhanced proprietary version developed and maintained by Siemens Product Lifecycle Management Software Inc.
- NVIDIA is a trademark or registered trademark of NVIDIA Corporation in the United States and/or other countries.
- OpenOffice.org and Apache OpenOffice are registered trademarks of the Apache Software Foundation.
- OpenSSL is a trademark owned by the OpenSSL Software Foundation.
- PSIM is a registered trademark of Powersim Inc.
- Python is a trademark or registered trademark of Python Software Foundation.
- ROMAX is a registered trademark of Romax Technology Limited, a subsidiary of Hexagon AB.
- Tecplot is a registered trademark of Tecplot, Inc. in the United States and other countries.
- PowerCore is a registered trademark of thyssenkrupp Electrical Steel GmbH.
- All other product and company names are trademarks, registered trademarks, or service marks of their respective companies, organizations, or holders.
- Note that [™] and [®] are not used in this document.

CONTENTS

User-Defined Template (API Functions and Samples)	5
API Functions	6
Notes	6
udt_application class	6
Panel and [Equations] Dialog	7
3D Geometry (Solid, Shell, Beam)	9
2D Geometry (Regions)	12
Operations	16
Obtain	25
Copy	27
Delete	29
Color	30
Transform	32
Obtain Distance	36
Determine Parallel or Perpendicular	37
Data	39
ID	43
Display	45
Error	45
udt_body_list class, udt_edge_list class, udt_face_list class, udt_vertex_list class	46
udt_curve_curve_int class	47
udt_position_vector class	48
udt_outcome class	49
udt_rgb_color class	49
Python Program Samples	51
Notes	51
Sample 1: Create Solid Template	51
Sample 2: Create Region	52
Sample 3: Create Basic Shape and Obtain Intersections	53
Sample 4: Python File Import	54
INDEX	55

User-Defined Template (API Functions and Samples)

It is important that users create their own Python programs by themselves when using user-defined region templates or user-defined solid templates.

This document will introduce both API functions usable within Python programs, as well as Python program samples.

■ *Topics in This Chapter*

- API Functions ([page 6](#))
- Python Program Samples ([page 51](#))

API Functions

- Notes ([page 6](#))
- udt_application class ([page 6](#))
- udt_body_list class, udt_edge_list class, udt_face_list class, udt_vertex_list class ([page 46](#))
- udt_curve_curve_int class ([page 47](#))
- udt_position_vector class ([page 48](#))
- udt_outcome class ([page 49](#))
- udt_rgb_color class ([page 49](#))

Notes

- API functions for user-defined templates are created using ACIS API functions as a basis. This includes covers and other items not available in Geometry Editor.
- The order of descriptions in the Python dictionary array is not critical to their understanding.
- Types used in arguments and return values are shown below.

Title in syntax	Type name	Description
string	String type	Takes string as argument and return value.
string array	String array type	Takes string array as argument and return value.
int	Integer type	Takes integer as argument and return value.
double	Real type	Takes real number as argument and return value.
variant	Variable type	Takes any type (string and integer, etc.) as argument and return value.
bool (true/false)	Logical type	Takes true/false as argument and return value.
object	Object type	Specifies object as argument and return value.
color	Color type	Takes color type (keywords such as “red”, or strings of “#RRGGBB” format) as argument and return value.
void	Void type	Shows return value type as “void” with methods without a return value.

udt_application class

- Panel and [Equations] Dialog ([page 7](#))
- 3D Geometry (Solid, Shell, Beam) ([page 9](#))
- 2D Geometry (Regions) ([page 12](#))
- Operations ([page 16](#))
- Obtain ([page 25](#))
- Copy ([page 27](#))
- Delete ([page 29](#))
- Color ([page 30](#))
- Transform ([page 32](#))

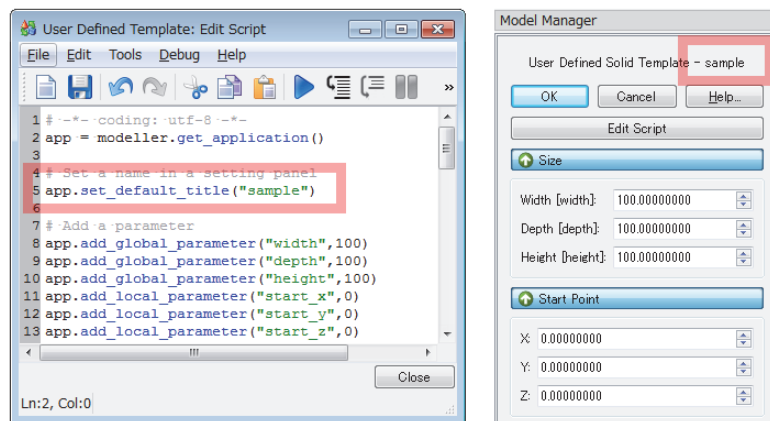
- Obtain Distance ([page 36](#))
- Determine Parallel or Perpendicular ([page 37](#))
- Data ([page 39](#))
- ID ([page 43](#))
- Display ([page 45](#))
- Error ([page 45](#))

Panel and [Equations] Dialog

■ ***set_default_title(const string & title)***

Sets a feature name. Title is displayed in a user-defined template feature panel.

Argument	Type	Description
title	string	Feature name



Example

■ ***add_local_parameter(const string& name, double value)***

Sets a variable and variable value to add to a user-defined template feature panel.

Argument	Type	Description
name	string	Variable name
value	double	Variable value

■ ***get_global_parameter(const string& name)***

Obtains value of a variable added to the [Equations] dialog.

Argument	Type	Description
name	string	Variable name

Return value type	Description
double	Variable value

■ ***get_local_parameter(const string& name)***

Obtains value of a variable added to a user-defined template feature

Argument	Type	Description
name	string	Variable name

Return value type	Description
double	Variable value

■ ***set_global_parameter_group(const string& name, const string& group)***

Sets the name of a group displayed in a panel. Value added to [Equations] dialog is used.

Argument	Type	Description
name	string	Variable name
group	string	Group name

■ ***set_global_parameter_label(const string& name, const string& label)***

Sets a label displayed in a panel. Value added to [Equations] dialog is used.

Argument	Type	Description
name	string	Variable name
value	string	Label name

■ ***set_local_parameter_group(const string& name, const string& group)***

Sets the name of a group box displayed in panel. Value added to a user-defined template feature panel is used.

Argument	Type	Description
name	string	Variable name
group	string	Group name

■ ***set_local_parameter_label(const string& name, const string& label)***

Sets a label displayed in panel. Value added to a user-defined template feature panel is used.

Argument	Type	Description
name	string	Variable name
value	string	Label name

3D Geometry (Solid, Shell, Beam)**■ *make_cuboid(const variant& value, udt_body* body)***

Creates a rectangular. With this function, specify width, depth, and height as arguments from start position.

Argument	Key	Type	Description
value (dictionary array)	width	double	Width
	depth	double	Depth
	height	double	Height
body	-	udt_body object	Solid created

Return value type	Description
udt_outcome object	Result of executed function

■ *solid_block(const variant& value, udt_body* body)*

Creates a rectangular. With this function, specify 2 points on a diagonal as arguments.

Argument	Key	Type	Description
value (dictionary array)	pt1	udt_position object	Vertex1 of solid
	pt2	udt_position object	Vertex 2 located on the diagonal point of the vertex 1
body	-	udt_body object	Solid created

Return value type	Description
udt_outcome object	Result of executed function

■ *make_sphere(const variant& value, udt_body* body)*

Creates a sphere. Specify radius with this function. The center of the solid created becomes the start position.

Argument	Key	Type	Description
value (dictionary array)	radius	double	Radius
body	-	udt_body object	Solid created

Return value type	Description
udt_outcome object	Result of executed function

■ *solid_sphere(const variant& value, udt_body* body)*

Creates a sphere. With this function, specify sphere center and radius.

Argument	Key	Type	Description
value (dictionary array)	center	udt_position object	Sphere center
	radius	double	Radius

Argument	Key	Type	Description
body	-	utd_body object	Solid created

Return value type	Description
udt_outcome object	Result of executed function

■ ***solid_cylinder_cone(const variant& value, udt_body* body)***

Creates a cylinder or cone.

Argument	Key	Type	Description
value (dictionary array)	pt1	udt_position object	Lower part center
	pt2	udt_position object	Upper part center
	major_radius	double	External lower part radius
	minor_radius	double	Internal lower part radius
	top_radius	double	Upper part radius
	xpt	udt_position object	Position of main direction. Specifying this key is optional.
body	-	udt_body object	Solid created

Return value type	Description
udt_outcome object	Result of executed function

■ ***make_torus(const variant& value, udt_body* body)***

Creates a torus. With this function, specify torus interior and exterior radius. The center of the torus created becomes the start position.

Argument	Key	Type	Description
value (dictionary array)	major_radius	double	Exterior radius
	minor_radius	double	Interior radius
body	-	udt_body object	Solid created

Return value type	Description
udt_outcome object	Result of executed function

■ ***solid_torus(const variant& value, udt_body* body)***

Creates a torus. With this function, specify torus center, exterior and interior radius.

Argument	Key	Type	Description
value (dictionary array)	center	udt_position object	Torus center
	major_radius	double	Exterior radius
	minor_radius	double	Interior radius
body	-	udt_body object	Solid created

Return value type	Description
udt_outcome object	Result of executed function

■ ***edge_helix(const variant& value, udt_edge* edge)***

Creates a helix.

Argument	Key	Type	Description
value (dictionary array)	axis_start	udt_position object	Start position of heliX-axis
	axis_end	udt_position object	End position of heliX-axis
	start_dir	udt_vector object	Helix start position direction
	radius	double	Radius
	thread_distance	double	Distance between threads along axis
	right_handed	bool	Twist direction True=1: Turns right False=0: Turns left
edge	-	udt_edge object	Edge created

Return value type	Description
udt_outcome object	Result of executed function

■ ***mk_polyhedron(const variant& value, udt_body* body)***

Creates regular polygon cylinder.

Argument	Key	Type	Description
value (dictionary array)	center	udt_position object	Center position
	start	udt_position object	Polygon 1st vertex
	sides	int	Number of polygon vertices
	height	double	Cylinder height
body	-	udt_body object	Solid created

Return value type	Description
udt_outcome object	Result of executed function

2D Geometry (Regions)

■ ***mk_ed_line(const udt_position* start, const udt_position* end, udt_edge* edge)***

Creates a line.

Argument	Type	Description
start	udt_position object	Start position
end	udt_position object	End position
edge	udt_edge object	Edge created

Return value type	Description
udt_outcome object	Result of executed function

■ ***curve_arc(const variant& value, udt_edge* edge)***

Creates an arc. With this function, specify arc center, radius, and start radius.

Argument	Key	Type	Description
value (dictionary array)	center	udt_position object	Arc center
	radius	double	Radius
	start_angle	double	Start angle
	end_angle	double	End angle
edge	-	udt_edge object	Edge created

Return value type	Description
udt_outcome object	Result of executed function

■ ***curve_arc_3pt(const variant& value, udt_edge* edge)***

Creates a circle or arc. With this function, specify 3 circle passing points (or 2 arc end points or passing points).

Argument	Key	Type	Description
value (dictionary array)	pt1	udt_position object	Pass point 1
	pt2	udt_position object	Pass point 2
	pt3	udt_position object	Pass point 3
	full	bool	True=1: Circle False=0: Arc
edge	-	udt_edge object	Edge created

Return value type	Description
udt_outcome object	Result of executed function

■ ***curve_arc_center_edge(const variant& value, udt_edge* edge)***

Creates an arc. With this function, specify arc center and 2 end points.

Argument	Key	Type	Description
value (dictionary array)	start	udt_position object	Arc center
	pt1	udt_position object	Arc end point 1
	pt2	udt_position object	Arc end point 2
	normal	udt_unit_vector object	Arc normal direction. Specifying this key is optional. udt_unit_vector contains length unit information. This does not need to be specified for 2D models.
edge	-	udt_edge object	Edge created

Return value type	Description
udt_outcome object	Result of executed function

■ ***curve_arc_diagonal(const variant& value, udt_edge* edge)***

Creates a circle or arc. With this function, specify 2 points on a circle (or on an arc) that face each other across the center.

Argument	Key	Type	Description
value (dictionary array)	pt1	udt_position object	Point 1 on a circle
	pt2	udt_position object	Point 2 on a circle
	full	bool	True=1: Circle False=0: Arc
edge	-	udt_edge object	Edge created

Return value type	Description
udt_outcome object	Result of executed function

■ ***mk_ed_ellipse(const variant& value, udt_edge* edge)***

Creates an elliptical arc.

Argument	Key	Type	Description
value (dictionary array)	center	udt_position object	Ellipse center
	normal	udt_unit_vector object	Ellipse normal direction. udt_unit_vector contains length unit information. This does not need to be specified for 2D models.
	major_axis	udt_vector object	Main axis direction
	radius_ratio	double	The ratio of main axis length to minor axis length
	start_angle	double	Start angle of arc with respect to main axis
	end_angle	double	End angle of arc with respect to main axis
edge	-	udt_edge object	Edge created

Return value type	Description
udt_outcome object	Result of executed function

■ ***edge_spiral(const variant& value, udt_edge* edge)***

Creates a spiral.

Argument	Key	Type	Description
value (dictionary array)	center	udt_position object	Spiral center
	normal	udt_vector object	Spiral normal direction. This does not need to be specified for 2D models.
	start_position	udt_position object	Start position
	width	double	Width
	angle	double	Angle
	right_handed	bool	Twist direction True=1: Turns right False=0: Turns left
edge	-	udt_edge object	Edge created

Return value type	Description
udt_outcome object	Result of executed function

■ ***function_curve(const variant& value, udt_edge* edge)***

Creates a function curve.

Argument	Key	Type	Description
value (dictionary array)	function_x	string	x function
	function_y	string	y function
	start	doublex (or t)	start value
	end	doublex (or t)	end value
edge	-	udt_edge object	Edge created

Return value type	Description
udt_outcome object	Result of executed function

■ ***make_ewire(const variant& value, udt_body* body)***

Connects edges and creates 1 region.

Argument	Key	Type	Description
value (dictionary array)	edges	udt_edge_list object	Connected edge array
body	-	udt_body object	Region created

Return value type	Description
udt_outcome object	Result of executed function

■ ***mk_region(const variant& value, udt_body* body)***

Creates a region.

There is no issue even if edges specified by the edges key are not connected. In this case, regions are created from the intersection of edges as vertices.

Argument	Key	Type	Description
value (dictionary array)	edges	udt_edge_list object	Edge array to create regions
	break_edges	bool	True =1: Creates region at intersection of edges False =0: Does not create region at intersection of edges
body	-	udt_body object	Region created

Return value type	Description
udt_outcome object	Result of executed function

■ ***mk_regular_polygon(const variant& value, udt_body* body)***

Creates a regular geometry edge.

Argument	Key	Type	Description
value (dictionary array)	center	udt_position object	Center position
	start	udt_position object	Polygon start point
	sides	int	Number of polygon vertices
body	-	udt_body object	Edge created

Return value type	Description
udt_outcome object	Result of executed function

Operations

■ ***intersect_curves(const variant& value, udt_curve_curve_int* cci)***

Obtains location of point where edges intersect.

Argument	Key	Type	Description
value (dictionary array)	edge1	udt_edge object	Edge 1
	edge2	udt_edge object	Edge 2
	filter	flag	Obtained intersecting point. • min_x: x direction minimum • min_y: y direction minimum • max_x: x direction maximum • max_y: y direction maximum Multiple points can be specified (Example: min_x min_y).
cci	-	udt_curve_curve_int object	Intersecting point

Return value type	Description
udt_outcome object	Result of executed function

■ ***split_curve(const variant& value, udt_edge_list* edge)***

Separated intersecting edges at an intersection.

Argument	Key	Type	Description
value (dictionary array)	edge	udt_edge object	Edge to separate
	split_pt	udt_position object	Separation location. When split_pt is specified, split_ed does not need to be specified.
	split_ed	udt_edge object	Edge to separate and intersecting edge. When split_ed is specified, split_pt does not need to be specified.
edge	-	udt_edge_list object	Edge after separation

Return value type	Description
udt_outcome object	Result of executed function

■ ***cover_wires(udt_body* body)***

Creates a covering face (cover) from 1 or multiple loops.

Argument	Type	Description
body	udt_body object	Loop (edge with one connection)

Return value type	Description
udt_outcome object	Result of executed function

■ ***cover_sheet(udt_body* body)***

Creates a covering face (cover) from 1 loop.

Argument	Type	Description
body	udt_body object	Loop (edge with one connection)

Return value type	Description
udt_outcome object	Result of executed function

■ ***uncover_face(udt_face* face)***

Removes a covering face (cover) from a loop edge.

Argument	Type	Description
face	udt_face object	Cover

Return value type	Description
udt_outcome object	Result of executed function

■ ***face_plane(const variant& value, udt_face* face)***

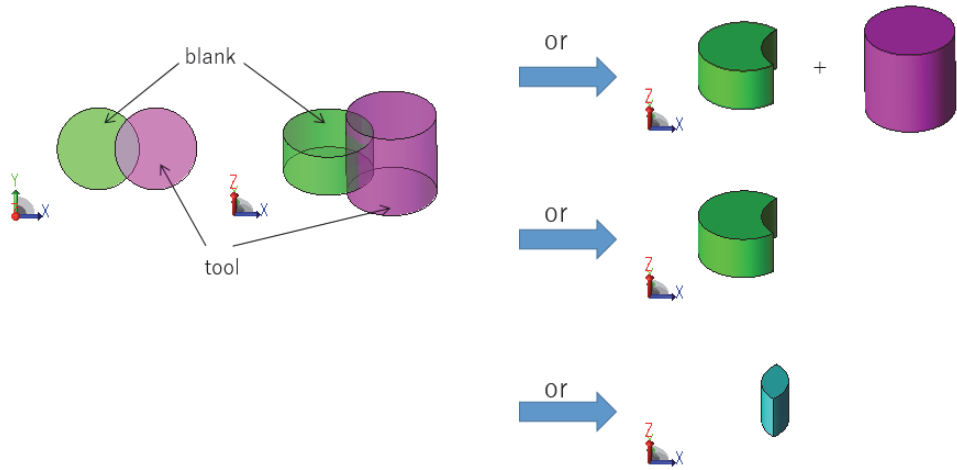
Creates a face.

Argument	Key	Type	Description
value (dictionary array)	p	udp_position object	Center position
	width	double	Horizontal length
	height	double	Vertical length
	normal	udt_vector object	Normal direction. This does not need to be specified for 2D models.
new_face	-	udt_face object	Face created

Return value type	Description
udt_outcome object	Result of executed function

■ ***boolean_chop_body(const variant& value)***

Separates a solid or region.



Argument	Key	Type	Description
value (dictionary array)	blank	udt_body object	Solid (region) for separation. Solids (or regions) specified as blank are divided by the solid specified as tool.
	tool	udt_body object	Solid (or region) to be separated.
	nonreg	bool	True=1: Normalized results required False=0: Normalized results not required
	outside	udt_body object	Solid (or region) as result of subtraction
	leftovers	udt_body object	Solid (or region) unrelated to separation.
	ndbool_keep	flag	Preserved state of solid (or region) for separation • ndbool_keep_neither: Keep neither • ndbool_keep_tool: Keep tool • ndbool_keep_blank: Keep blank • ndbool_keep_both: Keep both
	result_body	udt_body object	The solid (or region) where the result is stored if specifying anything other than ntbool_keep_neither as ndbool_keep.

Return value type	Description
udt_outcome object	Result of executed function

■ ***offset_body(const variant& value, udt_body* body)***

Moves a solid or region.

Argument	Key	Type	Description
value (dictionary array)	offset	double	Offset value
	box_low	udt_position object	Start position of intersection
	box_high	udt_position object	End position of intersection
	body	udt_face object	New solid

Return value type	Description
udt_outcome object	Result of executed function

■ ***sweep_with_options(const variant& value)***

Executes a sweep.

Argument	Key	Type	Description
value (dictionary array)	profile	udt_edge object or udt_body object	Sweep cross-section geometry edge
	path	udt_edge object or udt_body object	Sweep path
	distance	double	Distance
	vec	udt_vector object	Direction to sweep
	root	udt_position object	Point
	axis	udt_vector object	Axial direction

Return value type	Description
udt_outcome object	Result of executed function

MEMO Sweeps are executed by combinations of the below keys.

- profile, path
- profile, distance
- profile, vec
- profile, root, axis

MEMO Specify all keys to execute in the above order.

(Example: When profile, path and vec are specified, a sweep is executed via the combination of profile and path, and vec has no effect)

■ ***make_sweep_path(const variant& value, udt_edge* edge)***

Creates a path used for sweeps.

Argument	Key	Type	Description
value (dictionary array)	pts	udt_position_vector object	Path position array
edge	-	udt_edge object	New edge

Return value type	Description
udt_outcome object	Result of executed function

■ ***boolean(const variant& value)***

Executes a Boolean.

Argument	Key	Type	Description
value (dictionary array)	op	flag	Boolean types • union: Combination • intersection: Intersection part • subtraction: Intersection part deletion • nonreg_union: Irregular combination • nonreg_intersection: Irregular intersection part • nonreg_subtraction: Irregular intersection part deletion
	ndbool_keep	flag	Solid preserved state due to Boolean operations • ndbool_keep_neither: Keep neither • ndbool_keep_tool: Keep tool • ndbool_keep_blank: Keep blank • ndbool_keep_both: Keep both
	tool	utb_body object	Solid for Boolean
	blank	utb_body object	Solid with Boolean executed
	result_body	utb_body object	New solid if ndbool_keep_blank is selected with ndbool_keep.

Return value type	Description
udt_outcome object	Result of executed function

■ ***restore_entity_list(const variant& value, udt_body_list* body_list)***

Reads a SAT file.

Argument	Key	Type	Description
value (dictionary array)	file_namestr	ing	SAT file name
	text_mode	bool	True=1: Text mode False=0: Binary mode
body_list	-	udt_body_list object	New solid array

Return value type	Description
udt_outcome object	Result of executed function

■ ***sheet_from_ff(const variant& value, udt_body* body)***

Copies a face and creates a solid.

Argument	Key	Type	Description
value (dictionary array)	faces	udt_face_list object	Face copy array
body	-	udt_body object	Solid created

Return value type	Description
udt_outcome object	Result of executed function

■ ***sheet_thicken(const variant& value, udt_body* body)***

Creates a solid from a face.

Argument	Key	Type	Description
value (dictionary array)	thickness	double	Thickness
	both	bool	True=1: Creates a solid in both directions across a face False=0: Creates a solid in opposite directions across a face
	box_low	udt_position object	Solid lower face position
	box_high	udt_position object	Solid upper face position
	-	udt_body object	Solid created
body	-	udt_body object	Solid created

Return value type	Description
udt_outcome object	Result of executed function

■ ***separate_body(udt_body* body, udt_body_list* bodylist)***

Separates a solid or region.

Argument	Type	Description
body	udt_body object	Solid to separate
bodylist	udt_body_list object	Solid array to separate

Return value type	Description
udt_outcome object	Result of executed function

■ ***combine_body(udt_body* from_body, udt_body* to_body)***

Merges solids or regions.

Argument	Type	Description
from_body	udt_body object	Solid to merge
to_body	udt_body object	Solids or regions after merge

Return value type	Description
udt_outcome object	Result of executed function

■ ***blend_edges(const variant& value)***

Creates a fillet on the edge of a solid.

Argument	Key	Type	Description
value (dictionary array)	edges	udt_edge_list object	Edge of location to create fillet
	radius	double	Fillet radius

Return value type	Description
udt_outcome object	Result of executed function

■ ***chamfer_edges(const variant& value)***

Creates a chamfer on the edges of a solid.

Argument	Key	Type	Description
value (dictionary array)	edges	udt_edge_list object	Edge of location to create chamfer
	left_range	double	Left side chamfer width
	right_range	double	Right side chamfer width

Return value type	Description
udt_outcome object	Result of executed function

■ ***fillet_vertex(const QVariant& value)***

Creates a fillet on the vertex of a region.

Argument	Key	Type	Description
value (dictionary array)	vertex	udt_vertex object	Vertex of location to create fillet
	radius	double	Fillet radius

Return value type	Description
udt_outcome object	Result of executed function

■ ***imprint(udt_body* tool, udt_body* blank)***

Executes an imprint.

Argument	Type	Description
tool	udt_body object	Solid for imprint
blank	udt_body object	Solid imprinted

Return value type	Description
udt_outcome object	Result of executed function

■ ***move_faces(const variant& value)***

Moves a face.

Argument	Key	Type	Description
value (dictionary array)	faces	udt_face_list object	Array of faces for copying
	tr	udt_transf object	Transform settings
	box_low	udt_position object	Start position of intersection point used
	box_high	udt_position object	End position of intersection point used

Return value type	Description
udt_outcome object	Result of executed function

■ ***remove_face(udt_face* face)***

Deletes a face.

Argument	Type	Description
face	udt_face object	Face to delete

Return value type	Description
udt_outcome object	Result of executed function

■ ***remove_wire_edge(udt_edge* edge)***

Deletes an edge.

Argument	Type	Description
edge	udt_edge object	Edge to delete

Return value type	Description
udt_outcome object	Result of executed function

Obtain

■ ***get_faces(udt_body* body, udt_face_list* face_list)***

Obtains a solid from a face array.

Argument	Type	Description
body	udt_body object	Solid to be referenced
face_list	udt_face_list object	Face array obtained

Return value type	Description
udt_outcome object	Result of executed function

■ ***get_edges(udt_body* body, udt_edge_list* edge_list)***

Obtains a solid from an edge array.

Argument	Type	Description
body	udt_body object	Solid to be referenced
edge_list	udt_edge_list object	Edge array obtained

Return value type	Description
udt_outcome object	Result of executed function

■ ***get_edges(udt_face* face, udt_edge_list* edge_list)***

Obtains an edge array from a face.

Argument	Type	Description
face	udt_face object	Face to be referenced
edge_list	udt_edge_list object	Edge array obtained

Return value type	Description
udt_outcome object	Result of executed function

■ ***get_edges(udt_vertex* vertex, udt_edge_list* edge_list)***

Obtains an edge array from a vertex.

Argument	Type	Description
vertex	udt_vertex object	Vertex to be referenced
edge_list	udt_edge_list object	Edge array to be obtained

Return value type	Description
udt_outcome object	Result of executed function

■ ***get_vertex(udt_body* body, udt_vertex_list* vertex_list)***

Obtains a vertex array from a solid.

Argument	Type	Description
body	udt_body object	Solid to be referenced
vertex_list	udt_vertex_list object	Vertex array obtained

Return value type	Description
udt_outcome object	Result of executed function

■ ***get_vertex(udt_face* face, udt_vertex_list* vertex_list)***

Obtains a vertex array from a face.

Argument	Type	Description
face	udt_face object	Face to be referenced
vertex_list	udt_vertex_list object	Vertex array obtained

Return value type	Description
udt_outcome object	Result of executed function

■ ***get_vertex(udt_edge* edge, udt_vertex_list* vertex_list)***

Obtains a vertex array from an edge.

Argument	Type	Description
edge	udt_edge object	Edge to be referenced
vertex_list	udt_vertex_list object	Vertex array obtained

Return value type	Description
udt_outcome object	Result of executed function

Copy**■ *copy_body(udt_body* body, udt_body* new_body)***

Duplicates a solid.

Argument	Type	Description
body	udt_body object	Solid to duplicate
new_body	udt_body object	Solid created based on duplicates

Return value type	Description
udt_outcome object	Result of executed function

■ *copy_face(udt_face* face, udt_face* new_face)*

Duplicates a face.

Argument	Type	Description
face	udt_face object	Face to duplicate
new_face	udt_face object	Face created based on duplicates

Return value type	Description
udt_outcome object	Result of executed function

■ *copy_edge(udt_edge* edge, udt_edge* new_edge)*

Duplicates an edge.

Argument	Type	Description
edge	udt_edge object	Edge to duplicate
new_edge	udt_edge object	Edge created based on duplicates

Return value type	Description
udt_outcome object	Result of executed function

■ *copy_vertex(udt_vertex* vertex, udt_vertex* new_vertex)*

Duplicates a vertex.

Argument	Type	Description
vertex	udt_vertex object	Vertex to duplicate
new_vertex	udt_vertex object	Vertex created based on duplicates

Return value type	Description
udt_outcome object	Result of executed function

■ ***copy_body_list(udt_body_list* body_list, udt_body_list* copied_body_list)***

Duplicates a solid array.

Argument	Type	Description
body_list	udt_body_list object	Solid array to duplicate
copied_body_list	udt_body_list object	Solid array created based on duplicates

Return value type	Description
udt_outcome object	Result of executed function

■ ***copy_face_list(udt_face_list* face_list, udt_face_list* copied_face_list)***

Duplicates a face array.

Argument	Type	Description
face_list	udt_face_list object	Face array to duplicate
copied_face_list	udt_face_list object	Face array created based on duplicates

Return value type	Description
udt_outcome object	Result of executed function

■ ***copy_edge_list(udt_edge_list* edge_list, udt_edge_list* copied_edge_list)***

Duplicates an edge array.

Argument	Type	Description
edge_list	udt_edge_list object	Edge array to duplicate
copied_edge_list	udt_edge_list object	Edge array created based on duplicates

Return value type	Description
udt_outcome object	Result of executed function

■ ***copy_vertex_list(udt_vertex_list* vertex_list, udt_vertex_list* copied_vertex_list)***

Duplicates a vertex array.

Argument	Type	Description
vertex_list	udt_vertex_list object	Vertex array to duplicate
copied_vertex_list	udt_vertex_list object	Vertex array created based on duplicates

Return value type	Description
udt_outcome object	Result of executed function

Delete**■ *del_body(udt_body* given_body)***

Deletes a solid or region.

Argument	Type	Description
given_body	udt_body object	Solid to delete

Return value type	Description
udt_outcome object	Result of executed function

■ *del_face(udt_face* given_face)*

Deletes a face.

Argument	Type	Description
given_face	udt_face object	Face to delete

Return value type	Description
udt_outcome object	Result of executed function

■ *del_edge(udt_edge* given_edge)*

Deletes an edge.

Argument	Type	Description
given_edge	udt_edge object	Edge to delete

Return value type	Description
udt_outcome object	Result of executed function

■ *del_vertex(udt_vertex* given_vertex)*

Deletes a vertex.

Argument	Type	Description
given_vertex	udt_vertex object	Vertex to delete

Return value type	Description
udt_outcome object	Result of executed function

■ *del_body_list(udt_body_list* given_list)*

Deletes a solid array or region array.

Argument	Type	Description
given_list	udt_body_list object	Solid or region array to delete

Return value type	Description
udt_outcome object	Result of executed function

■ ***del_face_list(udt_face_list* given_list)***

Deletes a face array.

Argument	Type	Description
given_list	udt_face_list object	Face array to delete

Return value type	Description
udt_outcome object	Result of executed function

■ ***del_edge_list(udt_edge_list* given_list)***

Deletes an edge array.

Argument	Type	Description
given_list	udt_edge_list object	Edge array to delete

Return value type	Description
udt_outcome object	Result of executed function

■ ***del_vertex_list(udt_vertex_list* given_list)***

Deletes a vertex array.

Argument	Type	Description
given_list	udt_vertex_list object	Vertex array to delete

Return value type	Description
udt_outcome object	Result of executed function

Color

MEMO Main RGB colors are displayed below.

- Black: [0,0,0]
- Blue: [0,0,1]
- Green: [0,1,0]
- Cyan: [0,1,1]
- Red: [1,0,0]
- Magenta: [1,0,1]
- Yellow: [1,1,0]
- White: [1,1,1]

■ ***set_color(udt_body* body, udt_rgb_color* color)***

Sets the RGB color of a solid or region.

Argument	Type	Description
body	udt_body object	Solid or region
color	udt_rgb_color object	RGB color to set

Return value type	Description
udt_outcome object	Result of executed function

■ ***get_color(udt_body* body, udt_rgb_color* color)***

Obtains the RGB color of a solid or region.

Argument	Type	Description
body	udt_body object	Solid or region
color	udt_rgb_color object	RGB color

Return value type	Description
udt_outcome object	Result of executed function

■ ***rgb_color()***

Creates an RGB color.

There are no arguments.

Return value type	Description
udt_rgb_color	Result of executed function

■ ***rgb_color(double r, double g, double b)***

Creates an RGB color using three specified real numbers.

Argument	Type	Description
r	double	Red RGB color (from 0 to 1.0)
g	double	Green RGB color (from 0 to 1.0)
b	double	Blue RGB color (from 0 to 1.0)

Return value type	Description
udt_rgb_color	Result of executed function

Transform

■ ***translate_transf(udt_vector* vector)***

Creates an entity translation transformation.

Argument	Type	Description
vector	udt_vector object	Translation direction

Return value type	Description
udt_transf object	Transform settings

■ ***rotate_transf(double angle, udt_vector* vector)***

Creates an entity rotation transformation. This function can be used for 3D models.

The counterclockwise direction around the rotation axial direction is positive.

Argument	Type	Description
angle	double	Rotation angle
vector	udt_vector object	Rotation axis direction

Return value type	Description
udt_transf object	Transform settings

MEMO Use rotate_transf(double angle) for 2D models.

■ ***rotate_transf(double angle)***

Creates an entity rotation transformation. This function can be used for 2D models.

The Z-axis is used as the rotation axis. The counterclockwise direction around the Z-axis direction is positive.

Argument	Type	Description
angle	double	Rotation angle

Return value type	Description
udt_transf object	Transform settings

MEMO Use rotate_transf (double angle, udt_vector* vector) for 3D models.

■ ***coordinate_transf(udt_position* new_origin, udt_unit_vector* new_x_axis, udt_unit_vector* new_y_axis)***

Creates an entity coordinate transformation.

Argument	Type	Description
new_origin	udt_position object	Start position after transformation
new_x_axis	udt_unit_vector object	X-axis direction after transformation
new_y_axis	udt_unit_vector object	Y-axis direction after transformation

Return value type	Description
udt_transf object	Transform settings

■ ***scale_transf(double scale)***

Creates an entity scale transformation.

Argument	Type	Description
scale	double	Scaling factor

Return value type	Description
udt_transf object	Transform settings

■ ***reflect_transf(udt_unit_vector* axis)***

Defines a mirror plane. A mirror plane defined by this function can be used for copying solids and regions.

Argument	Type	Description
axis	udt_unit_vector object	Normal vector to a mirror plane. Mirror plane has normal vector specified here and passes through origin point.

Return value type	Description
udt_transf object	Transform settings

■ ***reflect_transf(udt_vector* axis)***

Defines a mirror plane. A mirror plane defined by this function can be used for copying solids and regions.

Argument	Type	Description
axis	udt_vector object	Normal vector to a mirror plane. Mirror plane has normal vector specified here and passes through origin point.

Return value type	Description
udt_transf object	Transform settings

■ ***apply_transf(udt_body* body, udt_transf* transf)***

Sets a solid or region transformation.

Argument	Type	Description
body	udt_body object	Solid or region to apply transformation
transf	udt_transf object	Transform settings

Return value type	Description
udt_outcome object	Result of executed function

■ ***apply_transf(udt_face* face, udt_transf* transf)***

Sets a transformation to a face.

Argument	Type	Description
face	udt_face object	Face to apply transformation
transf	udt_transf object	Transform settings

Return value type	Description
udt_outcome object	Result of executed function

■ ***apply_transf(udt_edge* edge, udt_transf* transf)***

Sets a transformation to an edge.

Argument	Type	Description
edge	udt_edge object	Edge to apply transformation
transf	udt_transf object	Transform settings

Return value type	Description
udt_outcome object	Result of executed function

■ ***apply_transf(udt_vertex* vertex, udt_transf* transf)***

Sets a transformation to a vertex.

Argument	Type	Description
vertex	udt_vertex object	Vertex to apply transformation
transf	udt_transf object	Transform settings

Return value type	Description
udt_outcome object	Result of executed function

■ ***transform_body(udt_body* body, udt_transf* transf)***

Transforms a solid or region.

Argument	Type	Description
body	udt_body object	Solid or region to apply transformation
transf	udt_transf object	Transform settings

Return value type	Description
udt_outcome object	Result of executed function

■ ***transform_face(udt_face* face, udt_transf* transf)***

Transforms a solid face.

Argument	Type	Description
face	udt_face object	Face to apply transformation
transf	udt_transf object	Transform settings

Return value type	Description
udt_outcome object	Result of executed function

■ ***transform_edge(udt_edge* edge, udt_transf* transf)***

Transforms an edge.

Argument	Type	Description
edge	udt_edge object	Edge to apply transformation
transf	udt_transf object	Transform settings

Return value type	Description
udt_outcome object	Result of executed function

■ ***transform_vertex(udt_vertex* vertex, udt_transf* transf)***

Transforms a vertex.

Argument	Type	Description
vertex	udt_vertex object	Vertex to apply transformation
transf	udt_transf object	Transform settings

Return value type	Description
udt_outcome object	Result of executed function

■ ***normalise(udt_vector* v)***

Converts from vector to unit_vector.

Argument	Type	Description
v	udt_vector object	Vertex

Return value type	Description
unit_vector object	Vector with length unit information

Obtain Distance

■ ***distance_to_line(udt_position* pt, udt_position* line_pt, udt_unit_vector* line_dir)***

Obtains the distance between a line and a point.

Argument	Type	Description
pt	udt_position object	Point position
line_pt	udt_position object	Line position
line_dir	udt_unit_vector object	Line direction

Return value type	Description
double	Distance

■ ***distance_to_plane(udt_position* pt, udt_position* plane_pt, udt_unit_vector* normal)***

Obtains the distance between a plane and a point.

Argument	Type	Description
pt	udt_position object	Point position
plane_pt	udt_position object	Face position
normal	udt_unit_vector object	Normal direction of plane

Return value type	Description
double	Distance

■ ***distance_to_point(udt_position* pt1, udt_position* pt2)***

Obtains the distance between a point and another point.

Argument	Type	Description
pt1	udt_position object	Point position 1
pt2	udt_position object	Point position 2

Return value type	Description
double	Distance

■ ***angle_between(udt_unit_vector* v1, udt_unit_vector* v2)***

Obtains the angle between a vector and another vector.

Argument	Type	Description
v1	udt_unit_vector object	Line vector 1. udt_unit_vector contains length unit information.
v2	udt_unit_vector object	Line vector 2. udt_unit_vector contains length unit information.

Return value type	Description
double	Angle

■ ***angle_between(udt_vector* v1, udt_vector* v2)***

Obtains the angle between a vector and another vector.

Argument	Type	Description
v1	udt_vector object	Line vector 1
v2	udt_vector object	Line vector 2

Return value type	Description
double	Angle

Determine Parallel or Perpendicular

■ ***antiparallel(udt_vector* v1, udt_unit_vector* v2)***

Obtains whether a vector and another vector are antiparallel.

Argument	Type	Description
v1	udt_vector object	Line vector 1
v2	udt_unit_vector object	Line vector 2 udt_unit_vector contains length unit information.

Return value type	Description
bool	True=1: Is antiparallel False=0: Is not antiparallel

■ ***antiparallel(udt_vector* v1, udt_vector* v2)***

Obtains whether a vector and another vector are antiparallel.

Argument	Type	Description
v1	udt_vector object	Line vector 1
v2	udt_vector object	Line vector 2

Return value type	Description
bool	True=1: Is antiparallel False=0: Is not antiparallel

■ ***antiparallel(udt_unit_vector* v1, udt_vector* v2)***

Obtains whether a vector and another vector are antiparallel.

Argument	Type	Description
v1	udt_unit_vector object	Line vector 1. udt_unit_vector contains length unit information.
v2	udt_vector object	Line vector 2

Return value type	Description
bool	True=1: Is antiparallel False=0: Is not antiparallel

■ ***antiparallel(udt_unit_vector* v1, udt_unit_vector* v2)***

Obtains whether a vector and another vector are antiparallel.

Argument	Type	Description
v1	udt_unit_vector object	Line vector 1. udt_unit_vector contains length unit information.
v2	udt_unit_vector object	Line vector 2. udt_unit_vector contains length unit information.

Return value type	Description
bool	True=1: Is antiparallel False=0: Is not antiparallel

■ ***are_parallel(udt_unit_vector* v1, udt_unit_vector* v2)***

Obtains whether a vector and another vector are antiparallel.

Argument	Type	Description
v1	udt_unit_vector object	Line vector 1. udt_unit_vector contains length unit information.
v2	udt_unit_vector object	Line vector 2. udt_unit_vector contains length unit information.

Return value type	Description
bool	True=1: Is parallel or antiparallel False=0: Is not parallel or antiparallel

■ ***are_parallel(udt_vector* v1, udt_vector* v2)***

Obtains whether a vector and another vector are antiparallel.

Argument	Type	Description
v1	udt_vector object	Line vector 1
v2	udt_vector object	Line vector 2

Return value type	Description
bool	True=1: Is parallel or antiparallel False=0: Is not parallel or antiparallel

■ ***are_perpendicular(udt_unit_vector* v1, udt_unit_vector* v2)***

Obtains whether a vector and another vector are perpendicular.

Argument	Type	Description
v1	udt_unit_vector object	Line vector 1. udt_unit_vector contains length unit information.
v2	udt_unit_vector object	Line vector 2. udt_unit_vector contains length unit information.

Return value type	Description
bool	True=1: Is perpendicular False=0: Is not perpendicular

■ ***are_perpendicular(udt_vector* v1, udt_vector* v2)***

Obtains whether a vector and another vector are perpendicular.

Argument	Type	Description
v1	udt_vector object	Line vector 1
v2	udt_vector object	Line vector 2

Return value type	Description
bool	True=1: Is perpendicular False=0: Is not perpendicular

Data

■ ***body()***

Obtains a solid or region.

Return value type	Description
udt_body object	Solid or region

■ ***face()***

Obtains a face.

Return value type	Description
udt_face object	Face

■ ***edge()***

Obtains an edge.

Return value type	Description
udt_edge object	Edge

■ ***vertex()***

Obtains a vertex.

Return value type	Description
udt_vertex object	Vertex

■ ***body_list()***

Obtains a solid or region array.

Return value type	Description
udt_body_list object	Solid or region array

■ ***face_list()***

Obtains a face array.

Return value type	Description
udt_face_list object	Face array

■ ***edge_list()***

Obtains an edge array.

Return value type	Description
udt_edge_list object	Edfe array

■ ***vertex_list()***

Obtains a vertex array.

Return value type	Description
udt_vertex_list object	Vertex array

■ ***position()***

Obtains a position.

Return value type	Description
udt_position object	Position

■ ***position(double x, double y, double z)***

Specifies coordinates (x, y, z), and obtains a position. Use this with 3D geometry.

Argument	Type	Description
x	double	Value of position x coordinates to obtain
y	double	Value of position y coordinates to obtain
z	double	Value of position z coordinates to obtain

Return value type	Description
udt_position object	Position

■ ***position(double x, double y)***

Specifies coordinates (x, y), and obtains a position. Use this with 2D geometry.

Argument	Type	Description
x	double	Value of position x coordinates to obtain
y	double	Value of position y coordinates to obtain

Return value type	Description
udt_position object	Position

■ ***vector()***

Obtains a vector.

Return value type	Description
udt_vector object	Vector

■ ***vector(double x, double y, double z)***

Specifies components (x, y, z), and obtains a position. Use this with 3D geometry.

Argument	Type	Description
x	double	Value of vector x component to obtain
y	double	Value of vector y component to obtain
z	double	Value of vector z component to obtain

Return value type	Description
udt_vector object	Vector

■ ***vector(double x, double y)***

Specifies components (x, y), and obtains a position. Use this with 2D geometry.

Argument	Type	Description
x	double	Value of vector x component to obtain
y	double	Value of vector y component to obtain

Return value type	Description
udt_vector object	Vector

■ ***unit_vector()***

Obtains a vector.

Return value type	Description
unit_vector object	Vector (with length unit information)

■ ***unit_vector(double x, double y, double z)***

Specifies components (x, y, z), and obtains a vector and its length unit information. Use this with 3D geometry.

Argument	Type	Description
x	double	Value of vector x component to obtain
y	double	Value of vector y component to obtain
z	double	Value of vector z component to obtain

Return value type	Description
unit_vector object	Vector (with length unit information)

■ ***unit_vector(double x, double y)***

Specifies components (x, y), and obtains a vector and its length unit information. Use this with 2D geometry.

Argument	Type	Description
x	double	Value of vector x component to obtain
y	double	Value of vector y component to obtain

Return value type	Description
unit_vector object	Vector (with length unit information)

■ ***position_vector()***

Obtains a position vector.

Return value type	Description
udt_position_vector object	Position vector

■ ***curve_curve_int()***

Obtains an intersection.

Return value type	Description
udt_curve_curve_int object	Intersection

ID■ ***set_persistent_id(udt_body* body, const string& id)***

Sets an ID to a solid or region.

Argument	Type	Description
body	udt_body object	Solid or region to set with ID
id	string	Solid or region ID

Return value type	Description
udt_outcome object	Result of executed function

■ ***set_persistent_id(udt_face* face, const string& id)***

Sets an ID to a face.

Argument	Type	Description
face	udt_face object	Face to set with ID
id	string	Face ID

Return value type	Description
udt_outcome object	Result of executed function

■ ***set_persistent_id(udt_edge* edge, const string& id)***

Sets an ID to an edge.

Argument	Type	Description
edge	udt_edge object	Edge to set with ID
id	string	Edge ID

Return value type	Description
udt_outcome object	Result of executed function

■ ***set_persistent_id(udt_vertex* vertex, const string& id)***

Sets an ID to a vertex.

Argument	Type	Description
vertex	udt_vertex object	Vertex to set with ID
id	string	Vertex ID

Return value type	Description
udt_outcome object	Result of executed function

■ ***get_persistent_id(udt_body* body)***

Obtains a solid or region ID.

Argument	Type	Description
body	udt_body object	Solid or region to obtain ID

Return value type	Description
string	Solid ID

■ ***get_persistent_id(udt_face* face)***

Obtains a face ID.

Argument	Type	Description
face	udt_face object	Face to obtain ID

Return value type	Description
string	Face ID

■ ***get_persistent_id(udt_edge* edge)***

Obtains an edge ID.

Argument	Type	Description
edge	udt_edge object	Edge to obtain ID

Return value type	Description
string	Edge ID

■ ***get_persistent_id(udt_vertex* vertex)***

Obtains a vertex ID.

Argument	Type	Description
vertex	udt_vertex object	Vertex to obtain ID

Return value type	Description
string	Vertex ID

Display

MEMO Call an API function such as `del_body()` to delete an object after executing `show()` or `show_preview()`. If an API function is not called, then the object will remain in memory.

■ ***show(udt_body* body)***

Displays entities.

Argument	Type	Description
body	udt_body object	Solid or region to display

There is no return value.

■ ***show(udt_body_list* list)***

Displays entities.

Argument	Type	Description
list	udt_body_list object	Solid array or region array to display

There is no return value.

■ ***show_preview(udt_edge_list* list)***

Previews geometry on the graphics window even while a panel is displayed.

Using this function, geometry can be displayed even if a script error has occurred. However, only an edge array (`udt_edge_list`) can be specified as an argument.

Argument	Type	Description
list	udt_edge_list object	Edge array to display

There is no return value.

Error

■ ***set_error(const string& message)***

Sets an error.

Argument	Type	Description
message	string	Error message

There is no return value.

■ ***set_warning(const string& message)***

Sets a warning.

Argument	Type	Description
message	string	Warning message

There is no return value.

udt_body_list class, udt_edge_list class, udt_face_list class, udt_vertex_list class

The `udt_body_list` class is used for operations related to solids. Similarly, the `udt_edge_list` class is used for operations related to solid edges, the `udt_face_list` class is used for operations related to solid faces, and the `udt_vertex_list` class is used for operations related to vertices.

The functions available in these four classes are the same. The below functions are explained using solids (body), but users should change these according to class.

■ ***add(const udt_body* body)***

Adds a solid (or region, face, edge, or vertex) to an array.

Argument	Type	Description
body (Or face, edge, or vertex)	udt_body object (or udt_edge object, udt_face object, udt_vertex object)	Solid (or region, face, edge, or vertex) to add to array

There is no return value.

■ ***at(int index)***

Deletes a position from an array.

Argument	Type	Description
index	int	Position index (starts from 0)

Return value type	Description
udt_position object	Position

■ ***count()***

Obtains the number of elements in an array

Return value type	Description
int	Number of elements in an array

■ ***first()***

Obtains the first solid (or region, face, edge, or vertex) in an array.

Return value type	Description
udt_body object, udt_edge object, udt_face object, or udt_vertex object	Solid, region, face, edge, or vertex

■ ***init()***

Initializes an array iterator.

There is no argument and return value.

■ ***next()***

Obtains the first solid (or region, face, edge, or vertex) in an array according to array iterator.

Return value type	Description
udt_body object, udt_edge object, udt_face object, or udt_vertex object	Solid, region, face, edge, or vertex

■ ***move_entity(udt_body* from)***

Replaces a specified solid (or region, face, edge, vertex) with another solid (or region, face, edge, vertex) in an array.

Argument	Type	Description
from	udt_body object (or udt_edge object, udt_face object, udt_vertex object)	The solid, region, face, edge, or vertex to be replaced

There is no return value.

■ ***is_null()***

Obtains whether a solid (or region, face, edge, vertex) exists via bool value.

Return value type	Description
bool	True=1: A solid, region, face, edge, or vertex exists. False=0: A solid, region, face, edge, or vertex does not exist.

udt_curve_curve_int class

The udt_curve_curve_int class is used for operations related to intersections.

■ ***int_point()***

Obtains an intersection position.

Return value type	Description
udt_position object	Position

■ ***is_null()***

Obtains whether is empty or not empty via bool value.

Return value type	Description
bool	True=1: Empty False=0: Not empty

■ ***next()***

Obtains the next intersection according to array iterator.

Return value type	Description
udt_curve_curve_int object	Intersection

udt_position_vector class

The udt_position_vector class is used for operations related to coordinate values and direction components.

■ ***at(int index)***

Deletes a position from an array.

Argument	Type	Description
index	int	Position index (starts from 0)

Return value type	Description
udt_position object	Position

■ ***clear()***

Initializes an array.

There is no argument and return value.

■ ***copy_to(udt_position_vector* pos_vec)***

Copies a specified array.

Argument	Type	Description
pos_vec	udt_position_vector object	Position vector

There is no return value.

■ ***push_back(udt_position* pos)***

Adds a position to the end of an array.

Argument	Type	Description
pos	udt_position object	Position

There is no return value.

■ ***resize(int new_size)***

Changes the number of elements in an array.

Argument	Type	Description
new_size	int	Array number of elements

There is no return value.

■ ***size()***

Deletes an element number from an array by integer value.

Return value type	Description
int	Array number of elements

udt_outcome class

The udt_outcome class is used when checking the results of executed functions.

■ ***error_message()***

Deletes an error message from a character string.

Return value type	Description
string	Error message string

■ ***ok()***

Obtains whether API is successful or not via bool value.

Return value type	Description
bool	True=1: Success False=0: Fail

udt_rgb_color class

The udt_rgb_color class is used for setting the color of a region (or solid, shell, or beam).

MEMO Main RGB colors are displayed below.

- Black: [0,0,0]
- Blue: [0,0,1]
- Green: [0,1,0]
- Cyan: [0,1,1]
- Red: [1,0,0]
- Magenta: [1,0,1]
- Yellow: [1,1,0]
- White: [1,1,1]

■ **blue()**

Obtains the RGB color blue value by real number.

Return value type	Description
double	Blue RGB color (from 0 to 1.0)

■ **green()**

Obtains the RGB color green value by real number.

Return value type	Description
double	Green RGB color (from 0 to 1.0)

■ **red()**

Obtains the RGB color red value by real number.

Return value type	Description
double	Red RGB color (from 0 to 1.0)

■ **set_blue(double b)**

Sets the RGB color blue value by real number.

Argument	Type	Description
b	double	Blue RGB color (from 0 to 1.0)

There is no return value.

■ **set_green(double g)**

Sets the RGB color green value by real number.

Argument	Type	Description
g	double	Green RGB color (from 0 to 1.0)

There is no return value.

■ **set_red(double r)**

Sets the RGB color red value by real number.

Argument	Type	Description
r	double	Red RGB color (from 0 to 1.0)

There is no return value.

Python Program Samples

- Notes ([page 51](#))
 - Sample 1: Create Solid Template ([page 51](#))
 - Sample 2: Create Region ([page 52](#))
 - Sample 3: Create Basic Shape and Obtain Intersections ([page 53](#))
 - Sample 4: Python File Import ([page 54](#))
-

Notes

- When starting the script, call “modeller.get_application()” and obtain the object to use for a user defined template.
-

Sample 1: Create Solid Template

This example is a template for creating a 3D rectangular. The start position and size of the rectangular are described to be specified in the geometry editor window.

```
app = modeller.get_application()

# Set a name in a panel
app.set_default_title("sample")

# Add a parameter
app.add_local_parameter("start_x",0)
app.add_local_parameter("start_y",0)
app.add_local_parameter("start_z",0)

# Set a parameter label in the panel
app.set_global_parameter_label("width","Width")
app.set_global_parameter_label("depth","Depth")
app.set_global_parameter_label("height","Height")
app.set_local_parameter_label("start_x","X")
app.set_local_parameter_label("start_y","Y")
app.set_local_parameter_label("start_z","Z")

# Set a group box in a panel
app.set_global_parameter_group("width","Size")
app.set_global_parameter_group("depth","Size")
app.set_global_parameter_group("height","Size")
app.set_local_parameter_group("start_x","Start Point")
app.set_local_parameter_group("start_y","Start Point")
app.set_local_parameter_group("start_z","Start Point")

# Obtain a parameter value
width = app.get_global_parameter("width")
depth= app.get_global_parameter("depth")
height= app.get_global_parameter("height")
start_x= app.get_local_parameter("start_x")
start_y= app.get_local_parameter("start_y")
start_z= app.get_local_parameter("start_z")

# Create a rectangular
# Use a python dictionary object
args = {"width":width, "depth":depth, "height":height}
box = app.body()
```

```
app.make_cuboid(args, box)

# Transform a rectangular
offset_x = (width/2) + start_x
offset_y = (height/2) + start_y
offset_z = (depth/2) + start_z
vector = app.vector(offset_x, offset_y, offset_z)
transf = app.translate_transf(vector)
app.apply_transf(box, transf)
app.show(box)
app.del_body(box)
```

Sample 2: Create Region

This example creates a region from 4 straight lines.

```
# function only (you need to call "app = modeller.get_application()")
def CreateRegion():

    # create line1
    start1 = app.position(-100,200)
    end1 = app.position(-100,-200)
    line1 = app.edge()
    ret = app.mk_ed_line(start1 , end1 , line1)
    if ret.ok() == False:
        app.set_error(ret.error_message())
        return False

    # create line2
    start2 = app.position(100,200)
    end2 = app.position(100,-200)
    line2 = app.edge()
    ret = app.mk_ed_line(start2 , end2 , line2)
    if ret.ok() == False:
        app.set_error(ret.error_message())
        return False

    # create line3
    start3 = app.position(-200,100)
    end3 = app.position(200,100)
    line3 = app.edge()
    ret = app.mk_ed_line(start3 , end3 , line3)
    if ret.ok() == False:
        app.set_error(ret.error_message())
        return False

    # create line4
    start4 = app.position(-200,-100)
    end4 = app.position(200,-100)
    line4 = app.edge()
    ret = app.mk_ed_line(start4 , end4 , line4)
    if ret.ok() == False:
        app.set_error(ret.error_message())
        return False

    # create region
    edges = app.edge_list()
    edges.add(line1)
    edges.add(line2)
    edges.add(line3)
    edges.add(line4)
```

```

# Use a python dictionary object
args = {'edges':edges, 'break_edges':True}
region = app.body()
ret = app.mk_region(args, region)
if ret.ok() == False:
    app.set_error(ret.error_message())
    return False

app.show(region)
app.del_body(region)

return True

```

Sample 3: Create Basic Shape and Obtain Intersections

This example creates straight lines and circles, and obtains the intersection of those as a udt_outcome object.

```

# function only (you need to call "app = modeller.get_application()")
def GetCrossPoint(edgelist):
    pos = []

    # create line
    start = app.position(-200,0)
    end = app.position(0,200)
    line = app.edge()
    ret = app.mk_ed_line(start, end, line)
    if ret.ok() == False:
        app.set_error(ret.message())
        return False, pos

    edgelist.add(line)

    # create circle
    center = app.position(0,0)
    circle = app.edge()
    circle_args = {'center':center, 'radius':150, 'start_angle':0, 'end_angle':0}
    ret = app.curve_arc(circle_args, circle)
    if ret.ok() == False:
        app.set_error(ret.message())
        return False, pos

    edgelist.add(circle)

    # get cross point
    # Use a python dictionary object
    args = {'edge1':line, 'edge2':circle, 'filter':'min_x|min_y'}
    cci = app.curve_curve_int()
    ret = app.intersect_curves(args, cci)
    if ret.ok() == False:
        app.set_error(ret.message())
        return False, pos

    while cci.is_null() == False:
        pos.append(cci.int_point())
        cci = cci.next()

    return True, pos

```

Sample 4: Python File Import

This example imports a file (Solid.py) containing a Python program.

By saving usable Python programs in a Python file (.py), they can then be edited with any editor and referenced from multiple user-defined template features.

■ **Solid.py**

```
def make_regular_polygon(app, body):
    center = app.position(0,0,0)
    start = app.position(0,100,0)
    args = {'center':center, 'start':start, 'sides':6, 'height':20}
    ret = app.mk_polyhedron(args, body)

    if ret.ok()==False:
        app.set_error(ret.error_message())
        return False

    return True
```

■ **Program for importing Solid.py into a user-defined template**

```
import sys
sys.path.append("//machine name/UDT/common")
import Solid

app = modeller.get_application()
body = app.body()
Solid.make_regular_polygon(app, body)
app.show(body)
app.del_body(body)
```

INDEX

A

add (const udt_body* body) 46
add_local_parameter (const string& name, double value) 7
angle_between (udt_unit_vector* v1, udt_unit_vector* v2) 36
angle_between (udt_vector* v1, udt_vector* v2) 37
antiparallel (udt_unit_vector* v1, udt_unit_vector* v2) 38
antiparallel (udt_unit_vector* v1, udt_vector* v2) 38
antiparallel (udt_vector* v1, udt_unit_vector* v2) 37
antiparallel (udt_vector* v1, udt_vector* v2) 37
apply_transf (udt_body* body, udt_transf* transf) 33
apply_transf (udt_edge* edge, udt_transf* transf) 34
apply_transf (udt_face* face, udt_transf* transf) 34
apply_transf (udt_vertex* vertex, udt_transf* transf) 34
are_parallel (udt_unit_vector* v1, udt_unit_vector* v2) 38
are_parallel (udt_vector* v1, udt_vector* v2) 39
are_perpendicular (udt_unit_vector* v1, udt_unit_vector* v2) 39
are_perpendicular (udt_vector* v1, udt_vector* v2) 39
at (int index) 46, 48

B

blend_edges (const variant& value) 23
blue () 50
body () 39
body_list () 40
boolean (const variant& value) 21
boolean_chop_body (const variant& value) 18

C

chamfer_edges (const variant& value) 23
clear () 48

combine_body (udt_body* from_body, udt_body* to_body) 23
coordinate_transf (udt_position* new_origin, udt_unit_vector* new_x_axis, udt_unit_vector* new_y_axis) 32
copy_body (udt_body* body, udt_body* new_body) 27
copy_body_list (udt_body_list* body_list, udt_body_list* copied_body_list) 28
copy_edge (udt_edge* edge, udt_edge* new_edge) 27
copy_edge_list (udt_edge_list* edge_list, udt_edge_list* copied_edge_list) 28
copy_face (udt_face* face, udt_face* new_face) 27
copy_face_list (udt_face_list* face_list, udt_face_list* copied_face_list) 28
copy_to (udt_position_vector* pos_vec) 48
copy_vertex (udt_vertex* vertex, udt_vertex* new_vertex) 27
copy_vertex_list (udt_vertex_list* vertex_list, udt_vertex_list* copied_vertex_list) 28
count () 46
cover_sheet (udt_body* body) 17
cover_wires (udt_body* body) 17
curve_arc (const variant& value, udt_edge* edge) 12
curve_arc_3pt (const variant& value, udt_edge* edge) 12
curve_arc_center_edge (const variant& value, udt_edge* edge) 13
curve_arc_diagonal (const variant& value, udt_edge* edge) 13
curve_curve_int () 43

D

del_body (udt_body* given_body) 29
del_body_list (udt_body_list* given_list) 29
del_edge (udt_edge* given_edge) 29
del_edge_list (udt_edge_list* given_list) 30
del_face (udt_face* given_face) 29
del_face_list (udt_face_list* given_list) 30
del_vertex (udt_vertex* given_vertex) 29
del_vertex_list (udt_vertex_list* given_list) 30

distance_to_line (udt_position* pt, udt_position*
line_pt, udt_unit_vector* line_dir) 36
distance_to_plane (udt_position* pt, udt_position*
plane_pt, udt_unit_vector* normal) 36
distance_to_point (udt_position* pt1, udt_position*
pt2) 36

E

edge () 40
edge_helix (const variant& value, udt_edge* edge)
11
edge_list () 40
edge_spiral (const variant& value, udt_edge* edge)
14
error_message () 49

F

face () 40
face_list () 40
face_plane (const variant& value, udt_face* face) 18
first () 46
function_curve (const variant& value, udt_edge*
edge) 15

G

get_color (udt_body* body, udt_rgb_color* color)
31
get_edges (udt_body* body, udt_edge_list*
edge_list) 25
get_edges (udt_face* face, udt_edge_list* edge_list)
25
get_edges (udt_vertex* vertex, udt_edge_list*
edge_list) 26
get_faces (udt_body* body, udt_face_list* face_list)
25
get_global_parameter (const string& name) 7
get_local_parameter (const string& name) 8
get_persistent_id (udt_body* body) 44
get_persistent_id (udt_edge* edge) 44
get_persistent_id (udt_face* face) 44
get_persistent_id (udt_vertex* vertex) 44
get_vertex (udt_body* body, udt_vertex_list*
vertex_list) 26
get_vertex (udt_edge* edge, udt_vertex_list*
vertex_list) 26

get_vertex (udt_face* face, udt_vertex_list*
vertex_list) 26
green () 50

I

init () 46
intersect_curves (const variant& value,
udt_curve_curve_int* cci) 16
int_point () 47
is_null () 47

M

make_cuboid (const variant& value, udt_body*
body) 9
make_ewire (const variant& value, udt_body* body)
15
make_sphere (const variant& value, udt_body*
body) 9
make_sweep_path (const variant& value, udt_edge*
edge) 21
make_torus (const variant& value, udt_body* body)
10
mk_ed_ellipse (const variant& value, udt_edge*
edge) 14
mk_ed_line (const udt_position* start, const
udt_position* end, udt_edge* edge) 12
mk_polyhedron (const variant& value, udt_body*
body) 11
mk_region (const variant& value, udt_body* body)
15
mk_regular_polygon (const variant& value,
udt_body* body) 16
move_entity (udt_body* from) 47
move_faces (const variant& value) 24

N

next () 47, 48
normalise (udt_vector* v) 35

O

offset_body (const variant& value, udt_body* body)
20
ok () 49

P

position () 41
 position (double x, double y, double z) 41
 position (double x, double y) 41
 position_vector () 42
 push_back (udt_position* pos) 48

R

red () 50
 reflect_transf (udt_unit_vector* axis) 33
 reflect_transf (udt_vector* axis) 33
 remove_face (udt_face* face) 24
 remove_wire_edge (udt_edge* edge) 25
 resize (int new_size) 49
 restore_entity_list (const variant& value,
 udt_body_list* body_list) 22
 rgb_color () 31
 rgb_color (double r, double g, double b) 31
 rotate_transf (double angle, udt_vector* vector) 32
 rotate_transf(double angle) 32

S

scale_transf (double scale) 33
 separate_body (udt_body* body, udt_body_list*
 bodylist) 23
 set_blue (double b) 50
 set_color (udt_body* body, udt_rgb_color* color)
 31
 set_default_title (const string & title) 7
 set_error (const string& message) 45
 set_global_parameter_group (const string& name,
 const string& group) 8
 set_global_parameter_label (const string& name,
 const string& label) 8
 set_green (double g) 50
 set_local_parameter_group (const string& name,
 const string& group) 8
 set_local_parameter_label (const string& name,
 const string& label) 8
 set_persistent_id (udt_body* body, const string& id)
 43
 set_persistent_id (udt_edge* edge, const string& id)
 43
 set_persistent_id (udt_face* face, const string& id)
 43

set_persistent_id (udt_vertex* vertex, const string&
 id) 43
 set_red (double r) 50
 set_warning (const string& message) 45
 sheet_from_ff (const variant& value, udt_body*
 body) 22
 sheet_thicken (const variant& value, udt_body*
 body) 22
 show (udt_body* body) 45
 show (udt_body_list* list) 45
 show_preview (udt_edge_list* list) 45
 size () 49
 solid_block (const variant& value, udt_body* body)
 9
 solid_cylinder_cone (const variant& value,
 udt_body* body) 10
 solid_sphere (const variant& value, udt_body* body)
 9
 solid_torus (const variant& value, udt_body* body)
 11
 split_curve (const variant& value, udt_edge_list*
 edge) 17
 sweep_with_options (const variant& value) 20

T

transform_body (udt_body* body, udt_transf*
 transf) 34
 transform_edge (udt_edge* edge, udt_transf* transf)
 35
 transform_face (udt_face* face, udt_transf* transf)
 35
 transform_vertex (udt_vertex* vertex, udt_transf*
 transf) 35
 translate_transf (udt_vector* vector) 32

U

udt_application class 6
 udt_body_list class 46
 udt_curve_curve_int class 47
 udt_edge_list class 46
 udt_face_list class 46
 udt_outcome class 49
 udt_position_vector class 48
 udt_rgb_color class 49
 udt_vertex_list class 46
 uncover_face (udt_face* face) 17
 unit_vector () 42

`unit_vector (double x, double y, double z)` [42](#)
`unit_vector (double x, double y)` [42](#)

V

`vector ()` [41](#)
`vector (double x, double y, double z)` [41](#)
`vector (double x, double y)` [42](#)
`vertex ()` [40](#)
`vertex_list ()` [40](#)